WEEK 01 TECHNICAL REPORT

YCB-Video Dataset Understanding I

Intern	Mehrana Rezaei
Group	Group 1 — Basic ML Knowledge
Main Quest	Neural Networks and Deep Learning
Side Quest	YCB-Video — Dataset Understanding I

1. Objective

The objective of Week 1 was to understand the YCB-Video dataset and its main annotation types before any model training. For this first-week study, I used the YCB-V BOP'19 test subset (approximately 660 MB) instead of downloading the full original dataset. The practical focus was on scenes and frames, RGB images, depth data, masks, bounding boxes, camera information, and 6D object pose annotations.

No model was trained during Week 1.

2. Main Quest Concepts

The assigned Main Quest for Week 1 was **Neural Networks and Deep Learning**. During this week, I studied the basic concepts behind binary classification and logistic regression, including how a logistic regression model produces predictions and how the cost function is used to measure prediction error. I also studied gradient descent and derivatives to understand how model parameters are updated in order to reduce the cost function.

I reviewed computation graphs and how derivatives can be calculated through them, then applied these ideas specifically to logistic regression. I also learned how gradient descent works when the model is trained on multiple examples instead of a single example. Another important part of the week was vectorization, where calculations are performed using vectors and matrices instead of explicit loops. This included vectorizing logistic regression and its gradient calculations.

In addition, I studied broadcasting in Python, reviewed common NumPy vector operations and shapes, and became more familiar with using Jupyter/IPython notebooks for machine-learning experiments. These topics provide the mathematical and programming foundation needed for the later stages of the YCB-Video project, especially when working with image arrays and eventually training neural networks and CNN-based models.

3. YCB-Video Background

YCB-Video is an RGB-D dataset introduced with PoseCNN for 6D object pose estimation in cluttered scenes. The dataset contains 21 YCB objects, 92 videos, and 133,827 frames. The original benchmark uses 640 × 480 images. In this notebook, I used the BOP-format test subset, which stores images as PNG files and annotations as JSON files.

3.1 BOP scene structure

- rgb/: color images for each frame.
- depth/: raw depth images corresponding to the RGB frames.
- mask/: full object masks.
- mask_visib/: the visible part of each object mask, useful when objects are occluded.

- scene_camera.json: camera intrinsic matrix and depth scale for each frame.
- scene_gt.json: object IDs and 6D ground-truth poses (rotation and translation).
- scene_gt_info.json: extra ground-truth information, including object bounding boxes.

4. Method

The dataset was downloaded programmatically from the BOP Benchmark Hugging Face repository. The notebook then inspected the test scenes, created a Pandas DataFrame to index RGB/depth paths and object counts, visualized representative RGB frames, and selected one frame for detailed analysis.

- Downloaded and extracted ycbv_base.zip and ycbv_test_bop19.zip.
- Loaded 12 BOP test scenes and built an index containing 900 RGB frames.
- Displayed the first eight RGB images from scene 000048.
- Loaded RGB and raw depth arrays for frame 000001.
- Read camera parameters from scene_camera.json.
- Read object IDs and 6D poses from scene_gt.json.
- Applied depth_scale to convert raw depth values to millimeters and centimeters.
- Inspected full and visible masks.
- Drew ground-truth bounding boxes from scene_gt_info.json.

5. Implementation

The implementation was completed in **week01_dataset_understanding.ipynb**. The notebook contains short Markdown explanations followed by executable inspection code. It uses Pathlib, JSON, NumPy, Pandas, Pillow, Matplotlib, and the Hugging Face Hub. There is no training loop.

Stage	Implementation	Purpose
Download	hf_hub_download + zipfile	Use the small BOP subset
Dataset index	Path.glob + json.load + Pandas	Create a table-like view of image records
RGB/depth	Pillow + NumPy	Inspect image shapes and values
Camera/pose	scene_camera.json + scene_gt.json	Understand geometric annotations
Masks	mask/ + mask_visib/	Inspect pixel-level object regions
Boxes	scene_gt_info.json + Matplotlib	Visualize object localization

6. Results

The executed notebook produced the following results for the BOP'19 test subset and the selected sample frame.

Result	Observed value
Test scenes	12
Indexed RGB frames	900
Selected scene / frame	000048 / 000001

RGB array	(480, 640, 3), uint8
Depth array	(480, 640), uint16
Raw depth range	0 to 28,800
Depth scale	0.1
Valid scaled depth	590–2,880 mm (59–288 cm)
Objects in selected frame	5 objects: IDs 1, 6, 14, 19, 20
Masks	5 full masks and 5 visible masks
Ground-truth bounding boxes	5
Camera intrinsic matrix	3 × 3
Object poses	5 poses; each uses a 3 × 3 rotation matrix + 3-value translation vector

6.1 Camera and pose example

Camera intrinsic matrix K for the selected frame:

1066.778	0	312.9869
0	1067.487	241.3109
0	0	1

The matrix contains the horizontal and vertical focal lengths (fx, fy), the principal point (cx, cy), zero skew terms, and the homogeneous-coordinate term. For object ID 1, the sample translation vector was approximately [-31.68, -17.37, 865.06] mm relative to the camera.

6.2 Representative visualizations

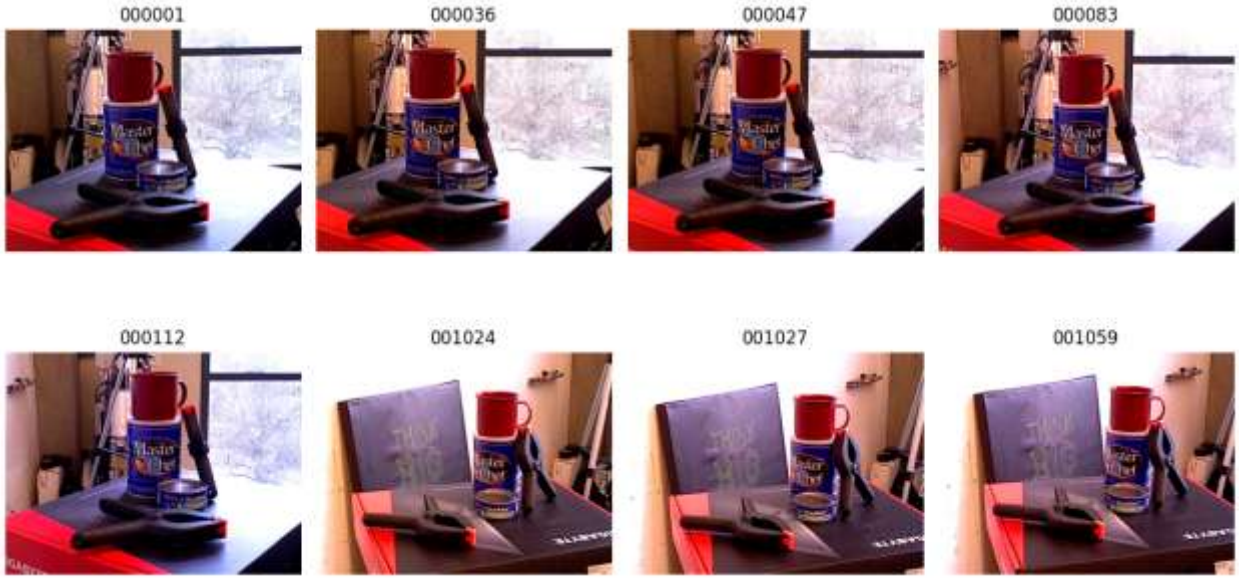


Figure 1. Eight RGB samples from scene 000048. Consecutive samples show viewpoint changes in a cluttered indoor scene.

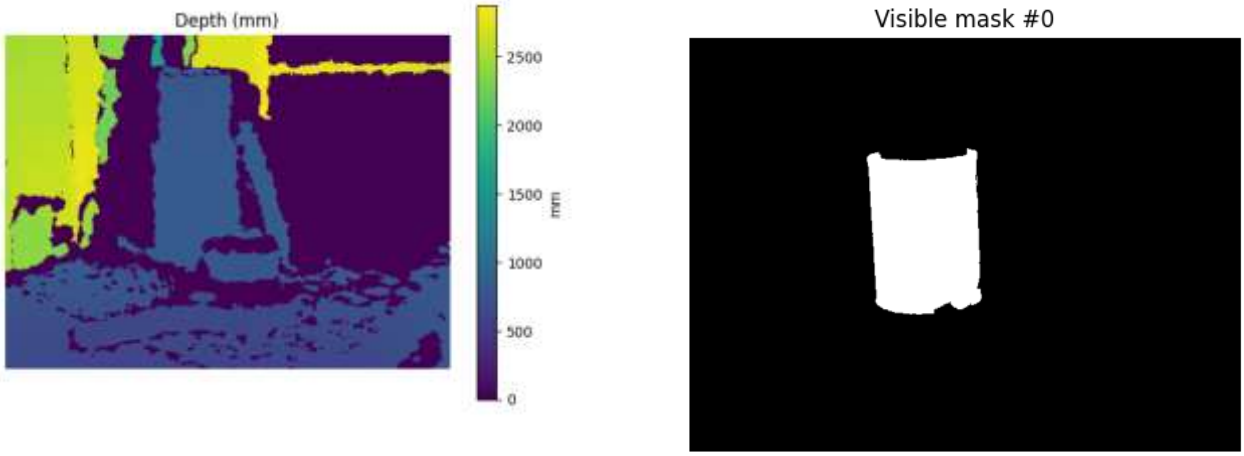


Figure 2. Scaled depth visualization (left) and one visible object mask (right).

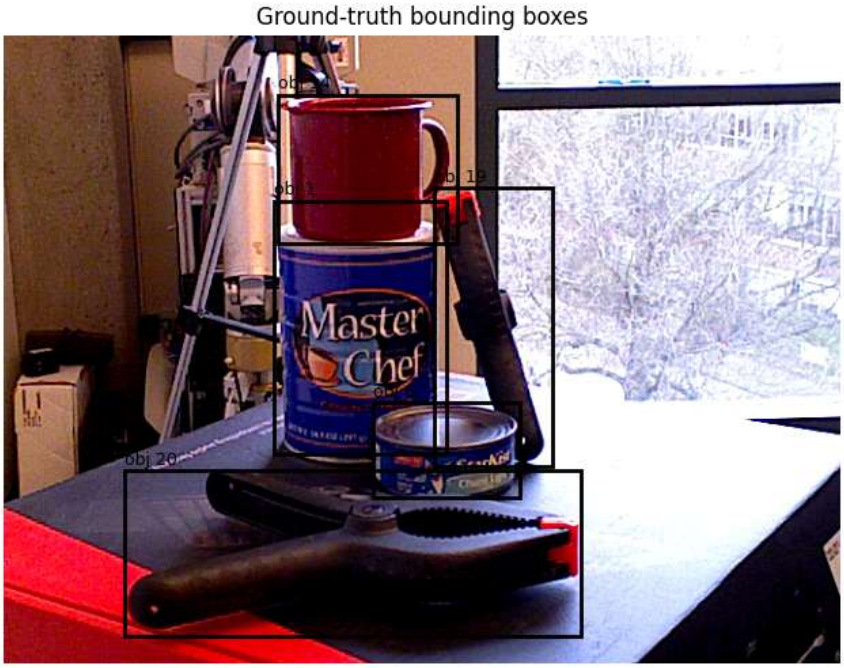


Figure 3. Ground-truth bounding boxes for the five annotated objects in frame 000001.

6.3 Observations

The RGB samples show a cluttered indoor tabletop scene containing several objects that overlap or partially occlude one another. The first test scene contains multiple viewpoints of the same group of objects, so neighboring frames can be visually similar while the camera/object viewpoint changes. This is important for later dataset splitting and robustness evaluation.

The selected RGB frame has a shape of 480 × 640 with three color channels, while the corresponding depth image has the same spatial resolution but only one depth value per pixel. After applying depth_scale = 0.1, the valid depth values ranged from 590 mm to 2,880 mm. The visible masks isolate only the pixels that can actually be seen, while the bounding boxes enclose each object with a rectangle and therefore also include some surrounding/background pixels. Five object annotations were consistently present in the selected frame across pose, mask, and bounding-box information.

7. Failure Analysis and Questions Discovered

No runtime failure is recorded in the final executed notebook. The main challenges in Week 1 were conceptual and data-format related rather than model-related. YCB-V in BOP format is not a single CSV table: images are stored as PNG files and annotations are spread across JSON files. I therefore built a Pandas DataFrame index from file paths and JSON metadata. I also had to understand how depth_scale converts raw depth values, how mask and mask_visib differ under occlusion, and how camera intrinsics and 6D pose matrices should be interpreted.

7.1 Questions for further study

1. How should the 3 × 3 rotation matrix be interpreted or visualized more intuitively for each object?
2. How should scenes be split later so that visually similar frames do not cause train/test leakage?
3. How can object IDs be mapped and summarized efficiently when building class-level statistics in Week 2?

8. Conclusion

During Week 1, I learned how the YCB-V BOP test data is organized and how one frame connects RGB, depth, mask, camera, pose, and bounding-box information. I verified the data by building an index of 900 test images, visualizing RGB and depth frames, reading camera and pose JSON files, inspecting full/visible masks, and drawing ground-truth bounding boxes. I also learned that file paths and structured annotation files can be combined into a DataFrame when a table-like overview is useful.